

AI AGENTS / API MANAGEMENT / MODEL CONTEXT PROTOCOL (MCP)

Designing APIs for agents

Learn how Webflow redesigned its APIs for AI agents, moving from endpoint wrapping to intent-driven Model Context Protocol design.

Aug 1st, 2026 10:00am by [Yan Xie](#), [Virat Patel](#) and [Albert Chang](#)



Karolina Grabowska for Unsplash+

[Webflow](#) sponsored this post.

In early 2025, Webflow started building for MCP before there was a clear playbook for agent-ready APIs. We publicly announced our MCP server in April 2025, then joined [Cloudflare's May 2025 Demo Day](#) alongside the first wave of remote MCP servers.

Our first approach was straightforward: wrap existing APIs as MCP tools. But we quickly learned that APIs designed for developers don't work well for agents. Developer APIs assume humans can read docs, do research, or get answers from

the API surface itself to guide planning and execution, and whether to retry, revise, or stop. So directly exposing Developer APIs to agents made our early MCP tools too low-level and overly chatty, often requiring many tool calls for simple tasks while struggling with more complex workflows. The result was subtle failures, inefficient runtimes, and unreliable task completion.

“APIs designed for developers don’t work well for agents. Developer APIs assume humans can read docs, do research, compose granular endpoints, manage state, and handle failures manually.”

Over the following year, we iterated heavily: redesigning tools around intents instead of endpoints, simplifying schemas, improving tool-calling efficiency, and making responses easier for agents to reason about. We also keep investing deeply in the underlying foundations – from the code layer to filesystem abstractions – to support more reliable and capable agent workflows.

Many of those decisions now resemble emerging industry patterns: task-oriented tools, agent-readable schemas, actionable error guidance, and APIs designed around intent rather than implementation details. This post shares the lessons that shaped **Webflow’s MCP server** and why designing for agents ultimately became much broader than exposing APIs through MCP. It required rethinking how autonomous systems discover capabilities, coordinate execution, navigate product surfaces, and operate reliably at runtime.

Why developer APIs don’t work well for agents

A user might ask an agent to update the homepage hero. In a traditional API, that can become a chain of low-level operations:



Webflow is the agentic web marketing platform where AI and teams design, build, optimize, and publish in one governed system. Visual authoring, structured CMS, managed hosting, and

[Learn more →](#)

THE LATEST FROM WEBFLOW

[5 things you can do with Webflow and the Claude connector](#)[Two AI paths to a Webflow Code Component](#)[Who owns the website?](#)

HEAR MORE FROM OUR SPONSOR

svo1975pvn1982@gmail.com[Submit](#)

```
list pages
→ find homepage
→ fetch page content
→ inspect content tree
→ find hero section
→ update content
```

Early MCP implementations often exposed existing developer APIs directly as MCP tools:

```
getPages()
getPageContent(pageId)
updateContent(contentId, payload)
publishPage(pageId)
```

MCP itself was not the problem. The protocol worked as intended. An LLM only sees tool descriptions, schemas, previous responses, and the current context window. It has to infer execution strategy dynamically while preserving intermediate state across multiple tool calls. That's where the failure modes begin to appear. Every additional tool call increases latency, token consumption, context pressure, and the probability of execution failure. The agent now has to:

TRENDING STORIES

3. 10 strategies to reduce MCP token bloat

4. Why the Model Context Protocol Won

5. APIs aren't dead. Here's where MCP fits alongside them.

- preserve intermediate identifiers correctly
- select the right resource from ambiguous results
- interpret noisy or overly generic responses
- coordinate dependent operations in sequence
- decide whether failures are retryable or terminal

Even when the API technically supports the workflow, too much of the model's runtime budget goes into navigating the API instead of completing the user's task.

What an agent API needs

The lesson for us was that exposing APIs through MCP was only the starting point. Reliable agent execution depended much more on the shape and semantics of the tools themselves. Developer APIs typically optimize for flexibility and composability. Agent APIs need to optimize for execution reliability: fewer ambiguous choices, fewer dependent calls, less state to preserve, and clearer guidance when something goes wrong.

“Developer APIs typically optimize for flexibility and composability. Agent APIs need to optimize for execution reliability: fewer ambiguous choices, fewer dependent calls, less state to preserve, and clearer guidance when something goes wrong.”

For the homepage hero update, the better interface is a task-level tool that matches the user's intent directly:

```
update_page_section({  
  page: "home",  
  section: "hero",
```

Declarative APIs turn complex, multi-step workflows into a single intent-driven call. Rather than requiring the agent to discover internal structures, preserve intermediate state, coordinate dependent operations, or recover from partial failures, the platform handles orchestration internally. This reduces failure modes and lets the agent focus on the user's goal instead of execution details.

Moving from low-level APIs to task-level tools was directionally correct, but it introduced a new problem at Webflow's scale: tool explosion. Webflow supports too many valid workflows to model each one as its own MCP tool. A purely task-oriented design quickly creates a large, overlapping tool surface that becomes difficult for agents to navigate efficiently. To solve this problem, there are basically 2 approaches, and we'll discuss both patterns in more depth in the sections that follow:

1. **A layered tool architecture:** Instead of exposing hundreds of standalone tools, we grouped related capabilities into broader domain-level tools with typed composable actions inside them. This gave agents a more structured and navigable map of the product surface while keeping the top-level tool space manageable.
2. **Introducing filesystem abstractions with code representations of Webflow projects:** Agents could operate against a structured project representation using familiar programming and shell-style workflows. This shifted part of the interaction model from API orchestration toward execution environments and code-driven manipulation.

Launching and scaling our MCP server

One of the lessons we learned was that designing APIs for agents extends beyond the API surface itself. Once agents became active participants in long-running workflows, many surrounding system decisions started to directly affect execution quality: session architecture, tool organization, runtime coordination, observability, and even distribution.

In practice, building reliable agent systems became a full-stack design problem. This shaped how we approached the **Webflow MCP server** across four areas:

- infrastructure for stateful execution and runtime coordination
- tool surface design for clarity and efficient capability discovery

1. Infrastructure

The infrastructure requirements for an MCP server turned out to be very different from a traditional API service. While MCP itself does not require server-side state, real-world agent workflows are highly stateful. An agent may authenticate once, inspect site context, call multiple tools across different product surfaces, and continue reasoning through a long-running workflow. Treating every interaction as fully stateless would force the model to repeatedly reconstruct context and execution state, increasing both latency and failure probability.

We built the Webflow MCP server on **Cloudflare** using **Durable Objects** because they provided a natural session-oriented execution model. Each MCP session owns its own server instance, managing user context, available tools, and runtime coordination without requiring separate session infrastructure.

We also had to support two different execution environments. Some operations run headlessly through **Webflow APIs**, while Designer actions — like inserting elements or updating styles — depend on the live browser canvas. For those workflows, the MCP server maintains a WebSocket bridge to a Designer Extension running inside the user's active Webflow session.

This architecture gives agents a unified MCP surface while allowing the system to route operations to the correct runtime behind the scenes. At the same time, we've been progressively moving more Designer capabilities into headless APIs. Reducing dependence on live browser execution simplifies the runtime architecture, lowers coordination complexity, and improves overall stability and reliability for agent workflows.

2. Designing the tool surface

As mentioned earlier, we found two different approaches that help reduce orchestration and discovery overhead for agents: layered tool organization and filesystem abstractions with code representations of Webflow projects. Over time, we started viewing them as short-term and long-term solutions to the same underlying problem.

In the short term, the layered tool architecture helps as the MCP surface expands as we give agents a better map rather than exposing all tools at the same time. Instead of exposing every task as a standalone MCP tool, we grouped related

needs, items, publishing, and updates, while Designer capabilities are organized around elements, styles, variables, assets, and components.

More specifically, the tool surface evolved into several layers:

- **Domain tools** organize broad product areas
- **Actions** provide composable typed primitives within each domain
- **Workflow tools** encapsulate orchestration-heavy tasks
- **Guide tools** provide operational instructions and product context for safer execution

While this approach definitely helped mitigate tool surface and orchestration issues, it does not fundamentally eliminate the problem. As product capabilities expand, the number of valid workflows and actions can still grow quickly, even within a layered architecture. At some point, the challenge shifts from simply organizing tools to reducing the system's dependence on explicit tool discovery altogether. Even well-structured tool hierarchies still require the model to search, select capabilities, interpret schemas, and coordinate execution across APIs.

This is where filesystem-style abstractions and code-oriented environments become increasingly important. Instead of forcing every interaction through explicit MCP tool invocation, agents can operate against a structured project representation more like a programmable workspace. **Anthropic** and **Cloudflare** have both discussed similar directions in recent agent infrastructure work. See the following example for a homepage update task with the code mode:

```
// agent edits your site as code
read src/pages/home.tsx
write src/components/Hero.tsx
write src/styles/theme.css
```

This plays much more naturally to current LLM strengths. Models are already highly effective at reasoning over structured text, generating code, editing files, and operating within execution environments. By shifting the interaction model from API orchestration toward structured state manipulation, the system reduces discovery overhead, context consumption, and multi-step coordination burden.

adding more tools. It is about giving models a more navigable and efficient operating environment.”

The deeper lesson is that scaling agent systems is not only about adding more tools. It is about giving models a more navigable and efficient operating environment.

3. Observability

Observability became much more important once agents started operating autonomously across long-running workflows.

Traditional logs and traces were useful for infrastructure debugging — they could tell us which requests failed, latency patterns, or server-side exceptions. But they were poorly suited for understanding agent behavior. They did not tell us what the agent was actually trying to accomplish, which capabilities it expected to exist, where workflows became confusing, or why seemingly “successful” sessions still produced poor outcomes.

To close that gap, we integrated MCPCat as an observability layer around the MCP server itself. By wrapping the server at initialization, it captures tool calls, resource reads, missing-tool attempts, session metadata, and response patterns across the full execution flow. The most valuable signal turned out to be intent.

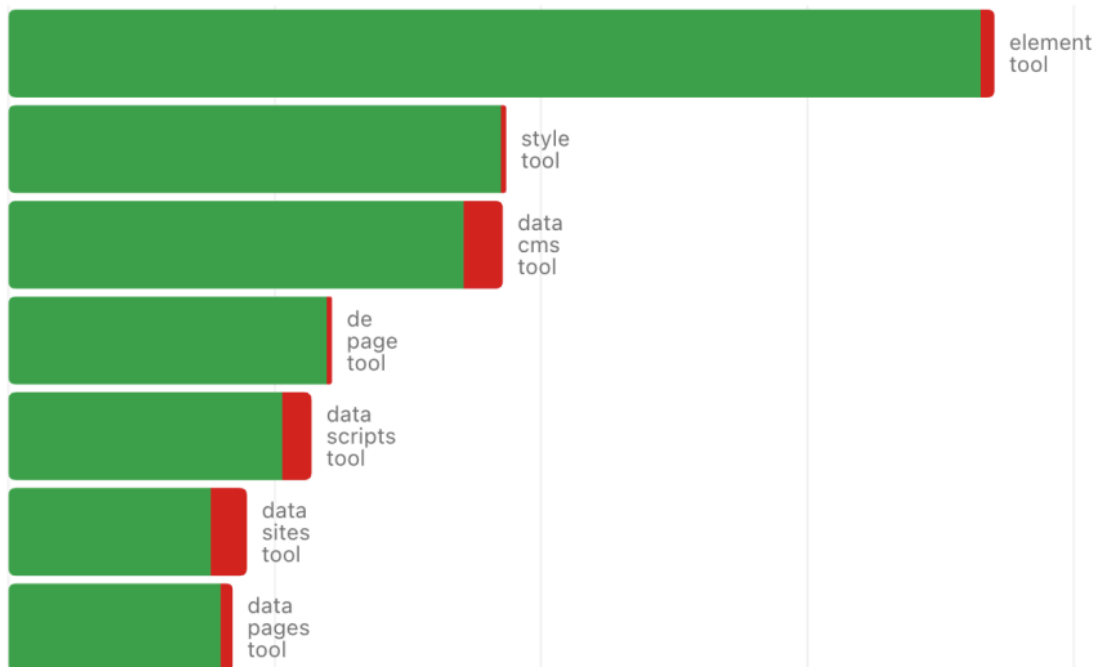
For the last 3 months



Canvas layout and structure	27.7%
CMS content management	16.9%
Custom code and interactions	14.1%
Project discovery	8.0%
Style and theme management	7.8%
SEO Optimization	7.0%
Responsive optimization	3.9%
CMS template management	3.7%
Localization management	2.5%
Visual styling & effects	1.9%
Other	6.4%

Most Used Tools

In the last 3 months



repeatedly explored, retried, or changed direction without ever producing a hard error. In many cases, the system appeared healthy from an infrastructure perspective while the agent experience was still failing silently.

We initially expected CMS migration and content management to dominate usage. Instead, Agent Goals showed that designing directly in the Webflow canvas through third-party agents accounted for 27.7% of sessions. We also discovered a silent error affecting more than 10% of sessions and found that 58.7% of sessions clustered around three core workflows.

Over time, we realized that **observability for agent systems** is fundamentally different from observability for APIs. The goal is not only understanding whether requests succeeded, but understanding how models reason, explore capabilities, recover from ambiguity, and fail during execution.

4. Scaling distribution

Scaling distribution for an MCP server was not only about handling more traffic. It was about reducing adoption friction, expanding where agents could use Webflow, and improving the quality of the workflows those agents could perform.

Adoption accelerated after we launched a hosted **MCP connector** in early 2026, starting with Claude, which significantly reduced setup friction for users. Instead of manually configuring MCP connections and authentication flows, users could connect to Webflow through a managed hosted integration with much less operational overhead. We've observed 6.7X session growth during the past 3 months.

Scaling adoption also made one thing clear: agents need more than tools. Tools expose what an agent can do; skills encode how to do it well. For Webflow, many successful outcomes require product judgment, reusable workflows, and domain conventions — not just API access. Bringing more useful skills into the Webflow MCP experience, such as layout patterns, CMS migration playbooks, SEO workflows, accessibility checks, and brand-system guidance, becomes critical as agents move from simple tool calls to end-to-end site work.

As adoption increased, the challenges shifted from protocol mechanics to product experience for autonomous systems: delivering richer site context, expanding the

Agents are not just another API client. They are a different kind of runtime participant: one that reasons through context, explores capabilities dynamically, learns from interface structure, and depends on the system to make the next correct action discoverable and reliable.

Designing for that kind of user changes much more than APIs. It affects how we structure tools, manage orchestration, expose context, build execution environments, observe workflows, and even how product surfaces themselves are modeled. Over time, we found ourselves designing not only MCP interfaces, but full **operating environments for autonomous systems**.

That pushed us toward layered tool architectures, workflow abstractions, filesystem-style project representations, richer observability, and skills layers. The goal was never simply exposing more capabilities to agents. It was helping models make correct decisions with less exploration, less orchestration burden, and greater execution reliability.

The work ahead is not just making Webflow programmable for agents. It is making Webflow legible, navigable, and operationally reliable for autonomous systems without losing the flexibility that makes the platform powerful for humans. That is both a deep product challenge and a systems challenge — and exactly the kind of problem we are excited to solve at Webflow.

This article was originally published on July 21, 2026, on [webflow.com](#).

TNS



Yan Xie is a Director of Engineering at Webflow, where she works on agentic product initiatives. She has 12+ years of experience across applied AI, product engineering, and monetization, with a focus on building products, systems, and teams that scale...

[Read more from Yan Xie →](#)



Virat Patel is a Software Engineer at Webflow, where he leads where Webflow MCP goes next and the growing ecosystem of AI agents around it. With 12+ years in and around the web, he specializes in the connective tissue between...

[Read more from Virat Patel →](#)



developers to build, test, and ship on Webflow, and now leads teams that...

[Read more from Albert Chang →](#)

TNS owner Insight Partners is an investor in: Anthropic.

SHARE THIS STORY



TRENDING STORIES

1. Designing APIs for agents
2. The MCP debate has a context problem
3. 10 strategies to reduce MCP token bloat
4. Why the Model Context Protocol Won
5. APIs aren't dead. Here's where MCP fits alongside them.

TNS DAILY NEWSLETTER

**Receive a free roundup of the
most recent TNS articles in
your inbox each day.**

svo1975pnv1982@gmail.com

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
Linux
Microservices
Open Source
Networking
Storage

OPERATIONS

AI Operations
CI/CD
Cloud Services
DevOps
Kubernetes
Observability
Operations
Platform Engineering

THE NEW STACK

About / Contact
Sponsors
Advertise With Us
Contributions

ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development
Large Language Models
Security
Software Development
WebAssembly

CHANNELS

Podcasts
Ebooks
Events
Webinars
Newsletter
TNS RSS Feeds



Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

FOLLOW TNS



© The New Stack 2026

[Disclosures](#)

[Terms of Use](#)

[Advertising Terms & Conditions](#)

[Privacy Policy](#)

[Cookie Policy](#)